

HeapSort (A) {

$\Rightarrow$ BuildHeap (A)

$\Rightarrow$ for (i $\leftarrow$ A.length downto 2) {

 swap (A, 1, i)

 A.heapsize $\leftarrow$ A.heapsize - 1

$\Rightarrow$ Heapify (A, 1)

 }

}

$$\boxed{O(n \cdot \log n)}$$

$$O(n) (n)$$

$$O(n) (n)$$

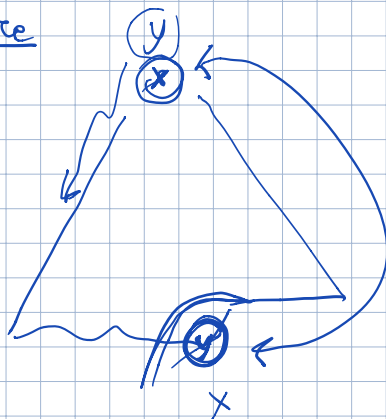
$$\left. \begin{array}{l} \text{Extract} \\ \leftarrow O(\log n) \end{array} \right\}$$

Caso migliore , caso peggiore HeapSort?

$O(n \log n)$ è sovrastimato?

È in-place? È stabile?

Caso Migliore



Se $x = y$

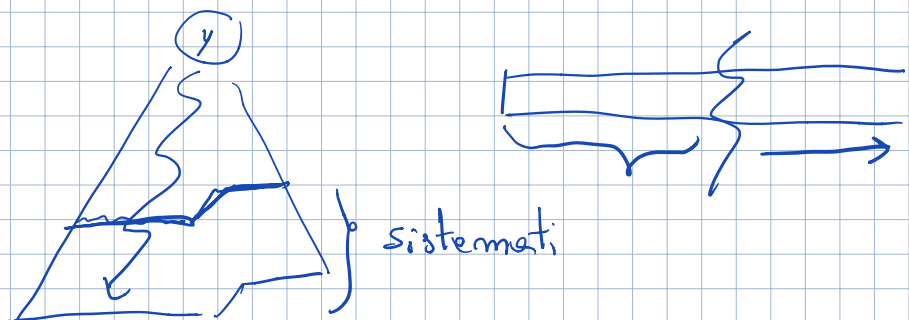
$\Rightarrow$
Heapify (A, 1)

$$O(1) (1)$$

Vettore con tutti
elementi uguali
 $\Rightarrow$

HeapSort $O(n) (n)$

Caso Peggior



L'altezza della heap su cui l'elemento decresce
di 1 dopo circa $\frac{n}{2}$ iterazioni

Se sulle prime $\frac{n}{2}$ iterazioni
Heapify richiede $O(\log n)$
allora HeapSort costa $O(n \log n)$

In place?

Heapify

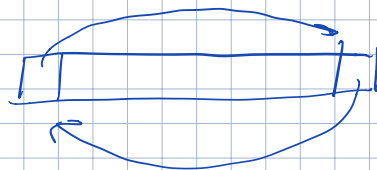
Ricorsiva di code

Eliminabile

Heapify non ricorsiva con un while

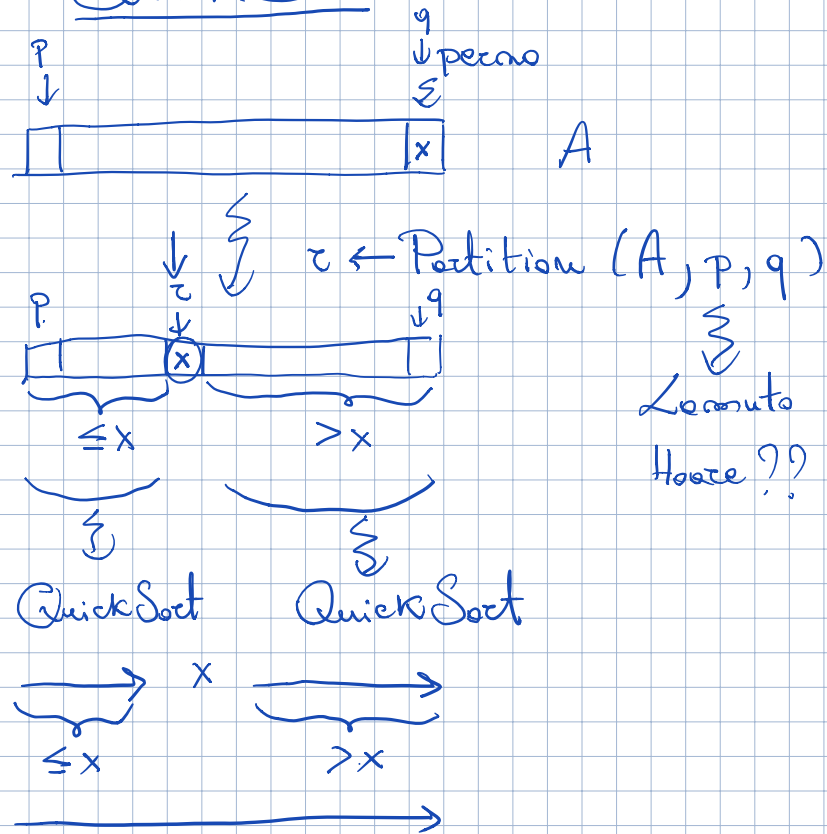
Sì HeapSort in-place

Stabile? No



QuickSort

Idea



QuickSort (A, p, q) {

if ($p < q$) {

$z \leftarrow \text{Partition}(A, p, q)$

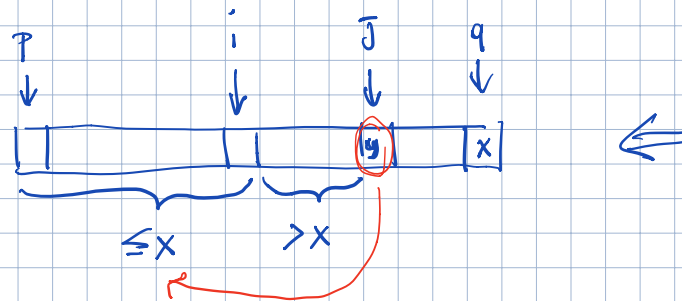
QuickSort ($A, p, z-1$)

QuickSort ($A, z+1, q$)

}

Partition

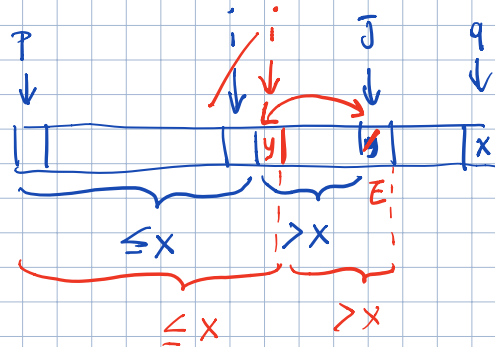
Idea



Sto migliorando $A[j] = y$

- se $y > x$ non faccio niente
al prossimo elemento guardo $A[j+1]$

- se $y \leq x$

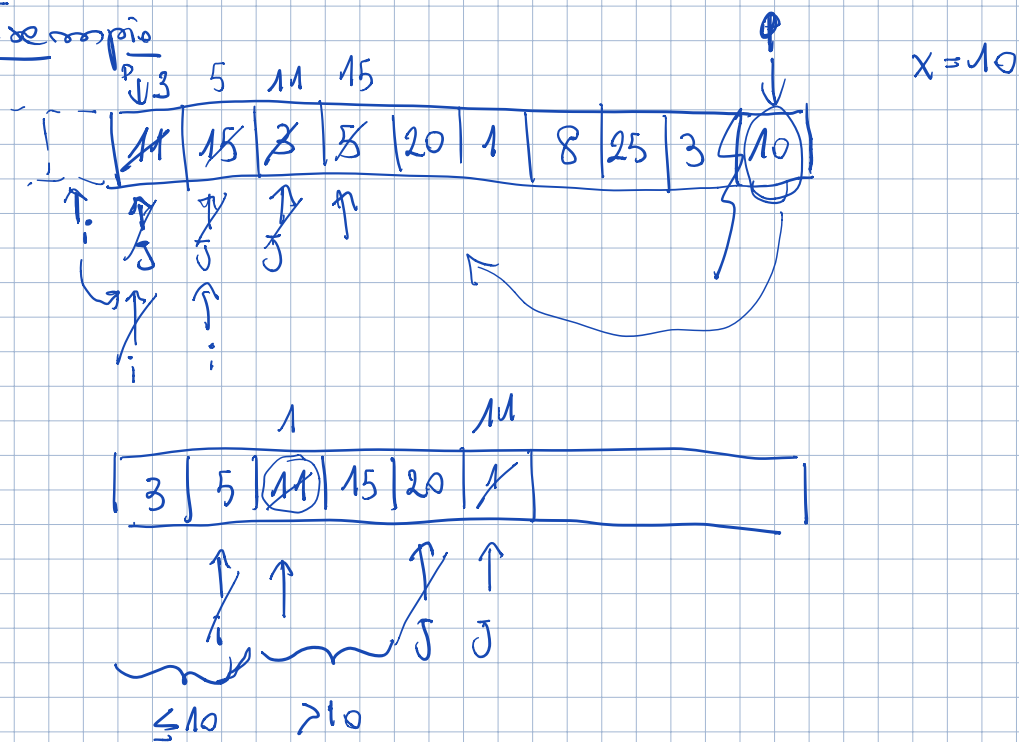


incremento i

scambio $A[i]$ con $A[j]$

al prossimo elemento guardo $A[j+1]$

Exemplo



Partition (A, p, q) $\{ \quad \} \quad \% m = q - p + 1$

$i \leftarrow p - 1$

$x \leftarrow A[q]$

for $(j \leftarrow p \text{ to } q) \{ \quad \}$ $\boxed{u(m)}$

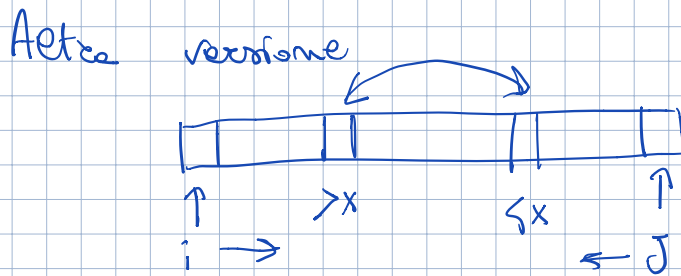
$\Rightarrow$ if $(A[j] \leq x) \{$

$i \leftarrow i + 1$

swap (A, i, j)

$\}$

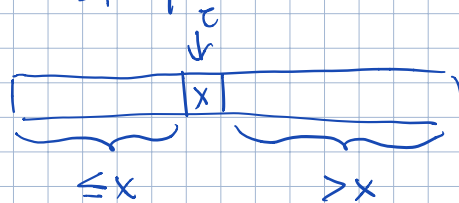
return i



- Complessità e Correttezza di Partition
- " " " Quick Sort
- In Place? Stabile?

Correttezza di Partition

Partition (A, p, q) termina ~~con~~ restituendo c tale che



$$A[p..c-1] \leq A[c] < A[c+1..q]$$

Invariante for

All' inizio della j -esima iterazione del for

$$A[p..i] \leq x < A[i+1..j-1]$$

Dim

Per induzione su J

BASE

$J=P$

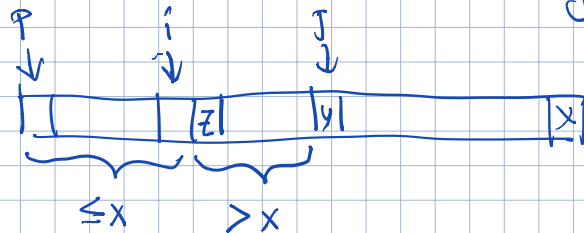
§) $A[p..p-1] \leq x < A[p..p-1]$

$\underbrace{\quad}_{\text{|| } \emptyset \checkmark} \quad \quad \quad \underbrace{\quad}_{\text{|| } \emptyset}$

✓

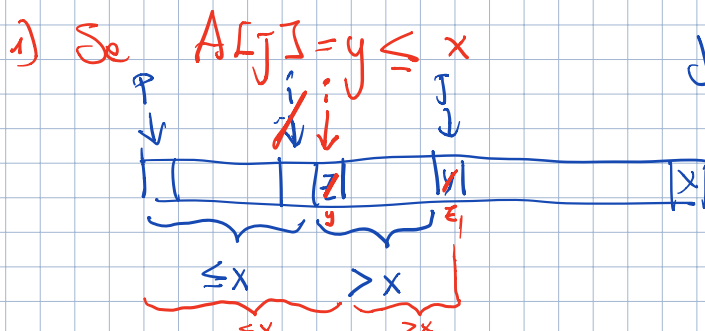
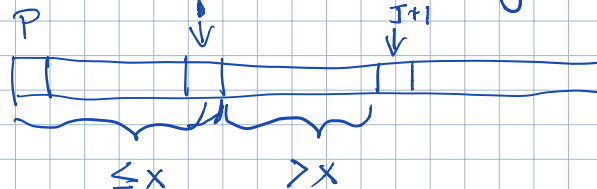
PASSO

Hplnd) Acc' inizio dell' it. J

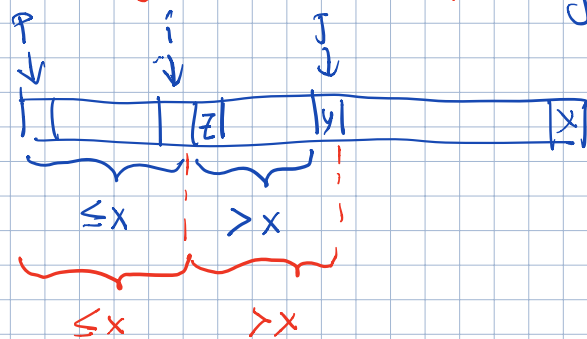


Esecuzione
della
 J -esima
iteraz.

§) Acc' inizio dell' it. $J+1$



2) Se $A[j] > x$ non faccio niente



✓

Correttezza QuickSort

QuickSort (A, p, q) termina con
 $A[p..q]$ ordinato

Dim.

Per induzione sul n. co. ch. ric.

BASE

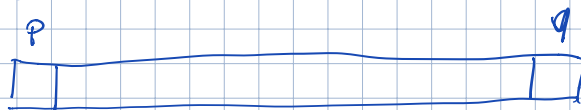
0 ch. ric.

$p \geq q$ $|A[p..q]| \leq 1$ ordinato

PASSO

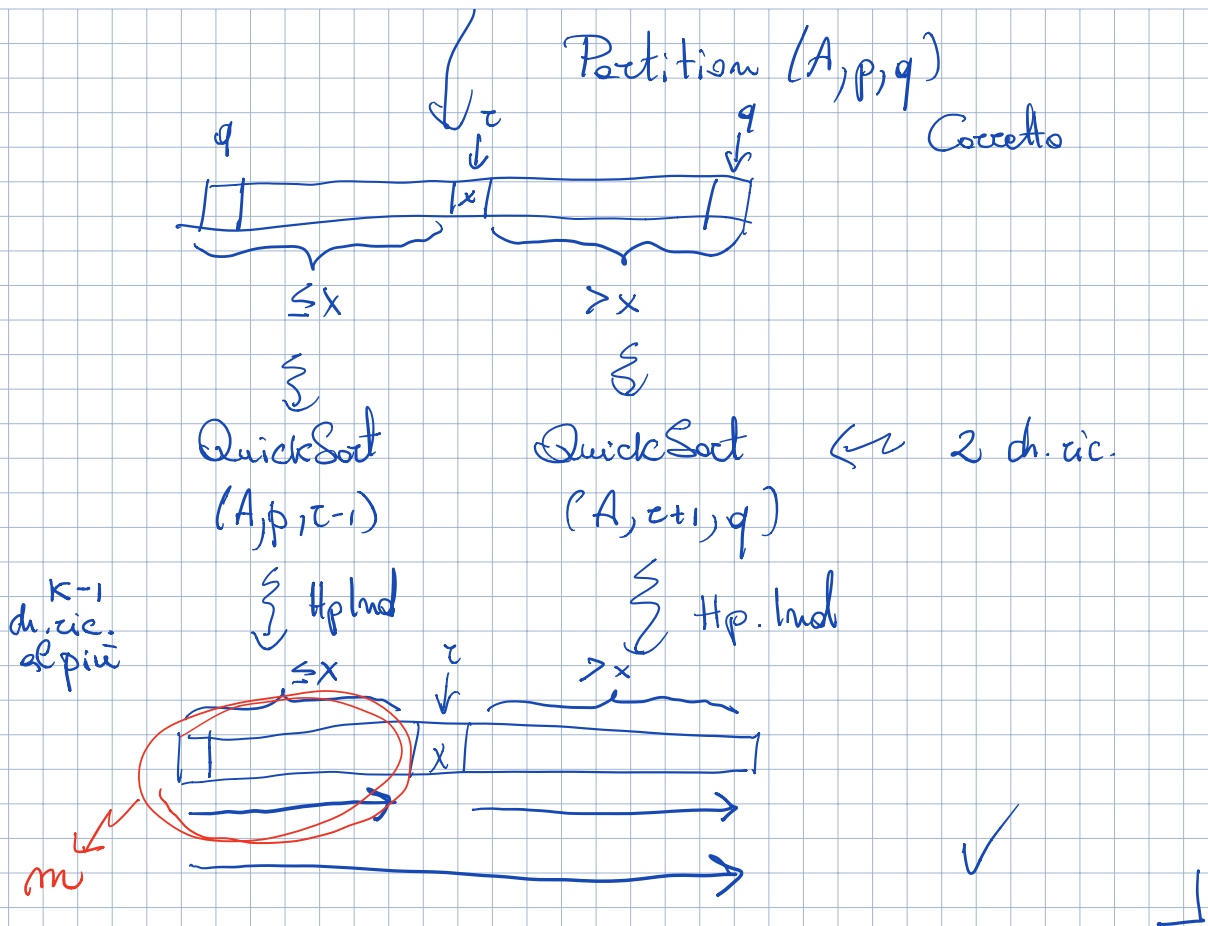
Hplud) $A[r_k^{\text{piu}}]$ ch. ric. ok

B) $k+1$ ch. ric. ok



$p < q$





$\text{QuickSort}(A, p, q) \{ T(n)$

if $(p < q) \{ \quad \leftarrow \Theta(1)$

$z \leftarrow \text{Partition}(A, p, q) \leftarrow \Theta(n)$

$\text{QuickSort}(A, p, z-1) \leftarrow T(m)$

$\text{QuickSort}(A, z+1, q) \leftarrow T(n-m-1)$

$\}$

$$T(n) = \begin{cases} \Theta(1) & n \leq 1 \\ T(m) + T(n-m-1) + \Theta(n) & n > 1 \end{cases}$$

$0 \leq m \leq n-1$

1. • Cerchiamo di indovinare una soluzione
- Per Sostituzione dimostriamo che è corretta

1. CASO PEGGIORE

$$m=0? \quad m=n-1?$$

$$T(n) = \begin{cases} \Theta(1) & n \leq 1 \\ T(n-1) + \Theta(n) & n > 1 \end{cases}$$

$$T(n) = \Theta(n^2)$$

$$m = \frac{1}{2}n?$$

$$T(n) = \begin{cases} \Theta(1) & n \leq 1 \\ 2T(\frac{n}{2}) + \Theta(n) & n > 1 \end{cases}$$

$$T(n) = \Theta(n \log n)$$

$$m = \frac{1}{100} m \quad \leadsto ? \quad \textcircled{u} \text{ (analogue)}$$

$$m = m - \underline{\underline{1000}} \quad \leadsto ? \quad \textcircled{u} (m^2)$$